

COMANDOS BÁSICOS DO GIT

Configuração Inicial

```
bash

# Configurar usuário
git config --global user.name "Seu Nome"
git config --global user.email "seu@email.com"

# Ver configurações

git config --list
```

Repositório Local

```
bash

# Inicializar repositório
git init

# Clonar repositório remoto
git clone <url-do-repositorio>

# Ver status dos arquivos
git status

# Adicionar arquivos ao staging
git add <arquivo>           # Arquivo específico
git add .                   # Todos os arquivos
git add *.js                # Por extensão

# Fazer commit
git commit -m "mensagem do commit"

# Ver histórico de commits
git log
git log --online           # Formato resumido

git log --graph            Com gráfico de branches
```

Trabalhando com Branches

```
bash
```

```
# Listar branches
git branch                # Locais
git branch -a             # Todas (locais e remotas)

# Criar branch
git branch <nome-branch>

# Mudar para branch
git checkout <nome-branch>
git switch <nome-branch>  # Comando mais moderno

# Criar e mudar para branch
git checkout -b <nome-branch>
git switch -c <nome-branch>

# Deletar branch
git branch -d <nome-branch> # Delete seguro

git branch -D <nome-branch> # Delete forçado
```

Integração com Remoto

```
bash

# Adicionar repositório remoto
git remote add origin <url>

# Enviar commits para remoto
git push origin <branch>
git push -u origin main    # Primeiro push com tracking

# Baixar alterações do remoto
git pull origin <branch>

# Buscar alterações sem merge
git fetch origin

# Sincronizar branch Local com remoto

git pull --rebase origin main
```



COMANDOS AVANÇADOS

Manipulação de Histórico

```
bash
```

```
# Modificar último commit
git commit --amend

# Reverter commit
git revert <hash-commit>

# Reset de arquivos
git reset <arquivo>      # Tira do staging
git reset --hard         # Descarta todas as alterações
git reset --soft <hash>  # Volta para commit específico

# Stash (guardar alterações temporariamente)
git stash                # Guardar alterações
git stash list           # Listar stashes
git stash apply          # Aplicar último stash
git stash pop            # Aplicar e remover

git stash drop           # Remover stash
```

Comparação e Merge

```
bash

# Ver diferenças
git diff                # Alterações não staged
git diff --staged       # Alterações staged
git diff <branch1> <branch2>

# Fazer merge
git merge <branch>

# Resolver conflitos (após merge com conflito)
# Editar arquivos com conflitos, depois:
git add <arquivos>

git commit
```

GIT FLOW

O que é Git Flow?

É uma metodologia de organização de branches para projetos. As branches principais são:

- main/master - Produção
- develop - Desenvolvimento
- feature/ - Novas funcionalidades
- release/ - Preparação para release
- hotfix/ - Correções urgentes

Instalação do Git Flow

```
bash

# Linux (Debian/Ubuntu)
sudo apt-get install git-flow

# macOS
brew install git-flow

# Windows (Git Bash)

# Geralmente já vem com Git for Windows
```

Inicializar Git Flow

```
bash

# Inicializar em repositório existente
git flow init

# Configuração padrão sugerida:
# - Branch production: main
# - Branch development: develop
# - Feature branches: feature/
# - Release branches: release/
# - Hotfix branches: hotfix/
# - Support branches: support/

# - Version tag prefix: (vazio)
```

Comandos Git Flow

Features (Funcionalidades)

```
bash

# Iniciar nova feature
git flow feature start <nome-feature>
```

```
# Publicar feature no repositório remoto
git flow feature publish <nome-feature>
```

```
# Finalizar feature (merge em develop)
git flow feature finish <nome-feature>
```

```
# Track feature existente
```

```
git flow feature track <nome-feature>
```

Releases (Versões)

```
bash
```

```
# Iniciar release
git flow release start <versao>
```

```
# Publicar release
git flow release publish <versao>
```

```
# Finalizar release (merge em main e develop + tag)
```

```
git flow release finish <versao>
```

Hotfixes (Correções Urgentes)

```
bash
```

```
# Iniciar hotfix
git flow hotfix start <versao>
```

```
# Finalizar hotfix
```

```
git flow hotfix finish <versao>
```

Fluxo Completo de Trabalho

```
bash
```

```
# 1. Iniciar nova funcionalidade
git flow feature start minha-feature
```

```
# 2. Desenvolver e commitar
git add .
git commit -m "Implementa funcionalidade X"
```

```
# 3. Publicar feature
git flow feature publish minha-feature
```

4. Finalizar feature

```
git flow feature finish minha-feature
```

5. Iniciar release quando várias features estiverem prontas

```
git flow release start 1.2.0
```

6. Finalizar release

```
git flow release finish 1.2.0
```

7. Enviar para repositório remoto

```
git push origin main
```

```
git push origin develop
```

```
git push --tags
```



COMANDOS ÚTEIS DO DIA A DIA

Limpeza e Manutenção

bash

Limpar arquivos não rastreados

```
git clean -n           # Mostra o que será removido
```

```
git clean -f           # Remove arquivos
```

```
git clean -fd          # Remove arquivos e diretórios
```

Otimizar repositório

```
git gc                 # Garbage collection
```

Ver espaço usado

```
git count-objects -v
```

Informações e Debug

bash

Ver quem modificou uma linha

```
git blame <arquivo>
```

Buscar no histórico

```
git log --grep="palavra"
```

Ver alterações específicas

```
git show <hash-commit>
```

Ver referências

```
git show-ref
```

Tags

```
bash
```

```
# Criar tag
```

```
git tag <nome-tag>
```

```
git tag -a v1.0 -m "Versão 1.0"
```

```
# Listar tags
```

```
git tag
```

```
# Enviar tags para remoto
```

```
git push --tags
```



DICAS PRÁTICAS

1. Commits frequentes - Faça commits pequenos e descritivos
2. Branches descritivas - Use nomes claros para branches
3. Pull antes do push - Sempre sincronize antes de enviar
4. Convenção de commits - Use mensagens no padrão:
5. text

```
feat: adiciona nova funcionalidade
```

```
fix: corrige bug no login
```

6. docs: atualiza documentação

Este conjunto de comandos cobre a maioria das necessidades no desenvolvimento

diário com Git! 🚀